Advanced Software Engineering VU [194.187]

Maria Christakis

TU Wien

https://mariachris.github.io

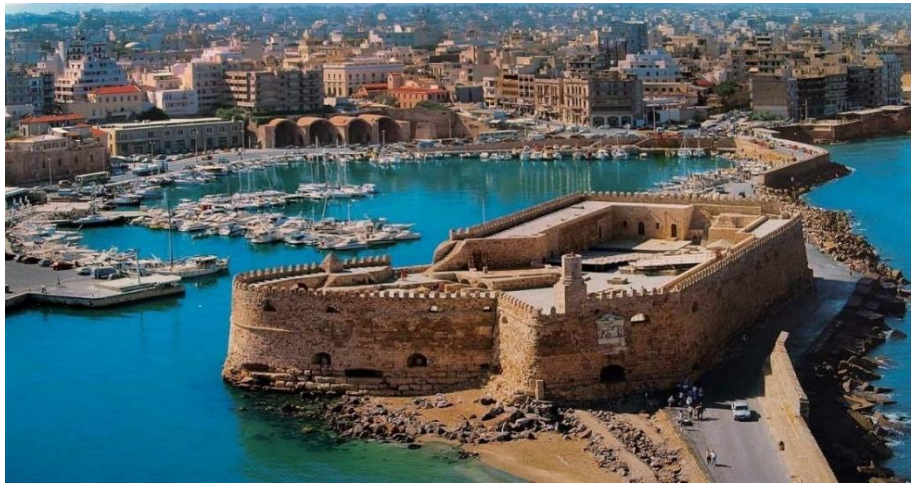
About me

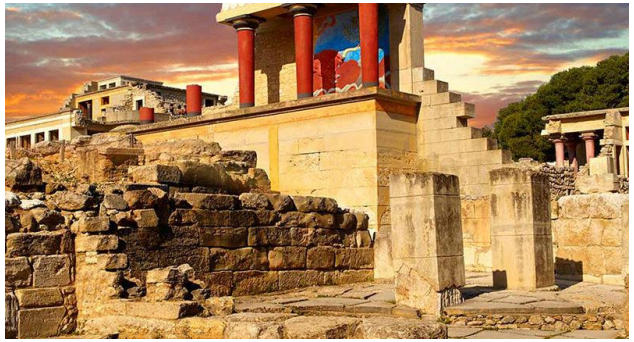
I am from Crete, Greece



I am from Crete, Greece



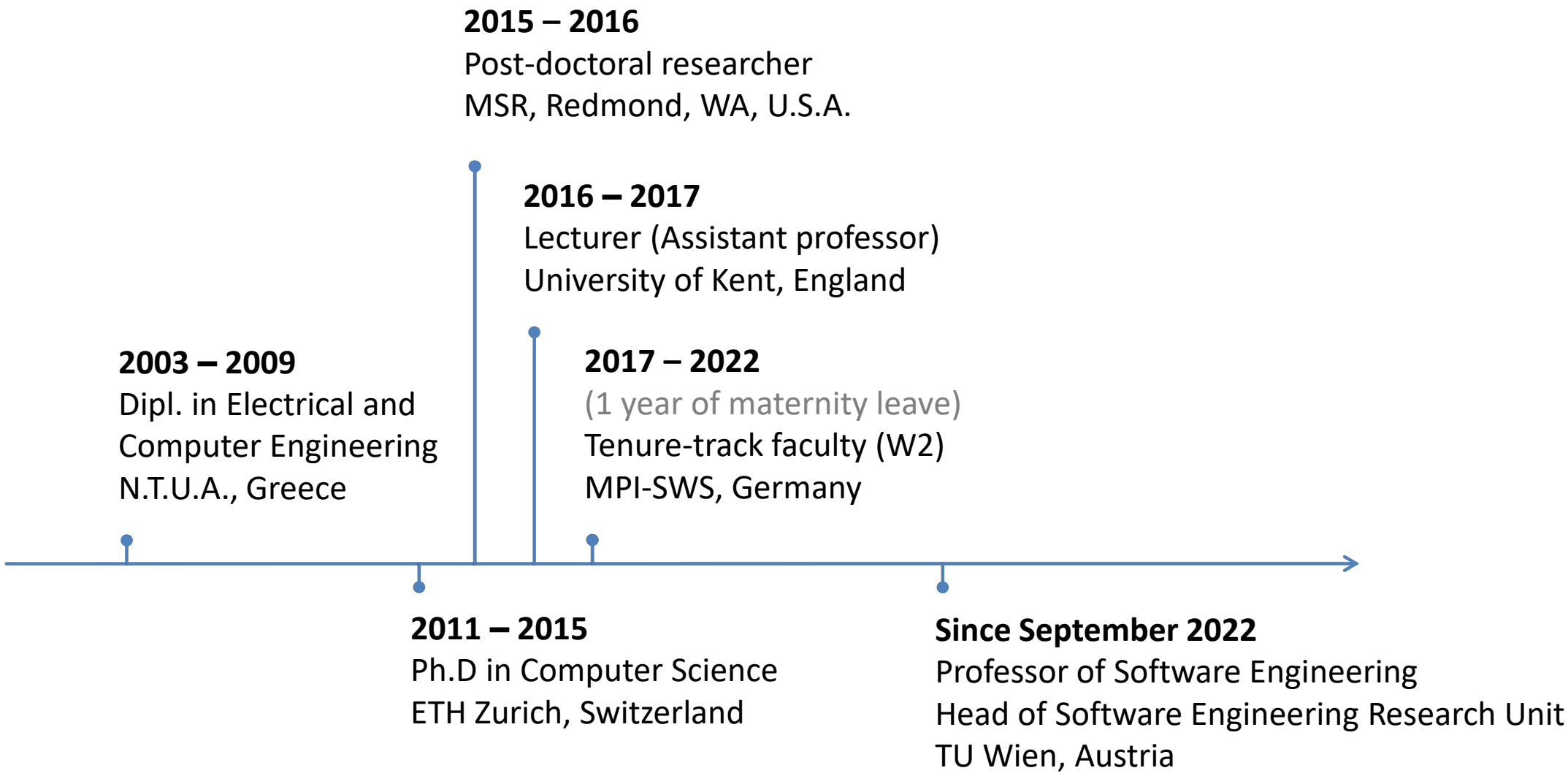








Where I have been until now



About my research

Software has bugs

Software is everywhere

but it is not reliable









Program analyzers find software bugs

A program analyzer is software that analyzes other software to find bugs



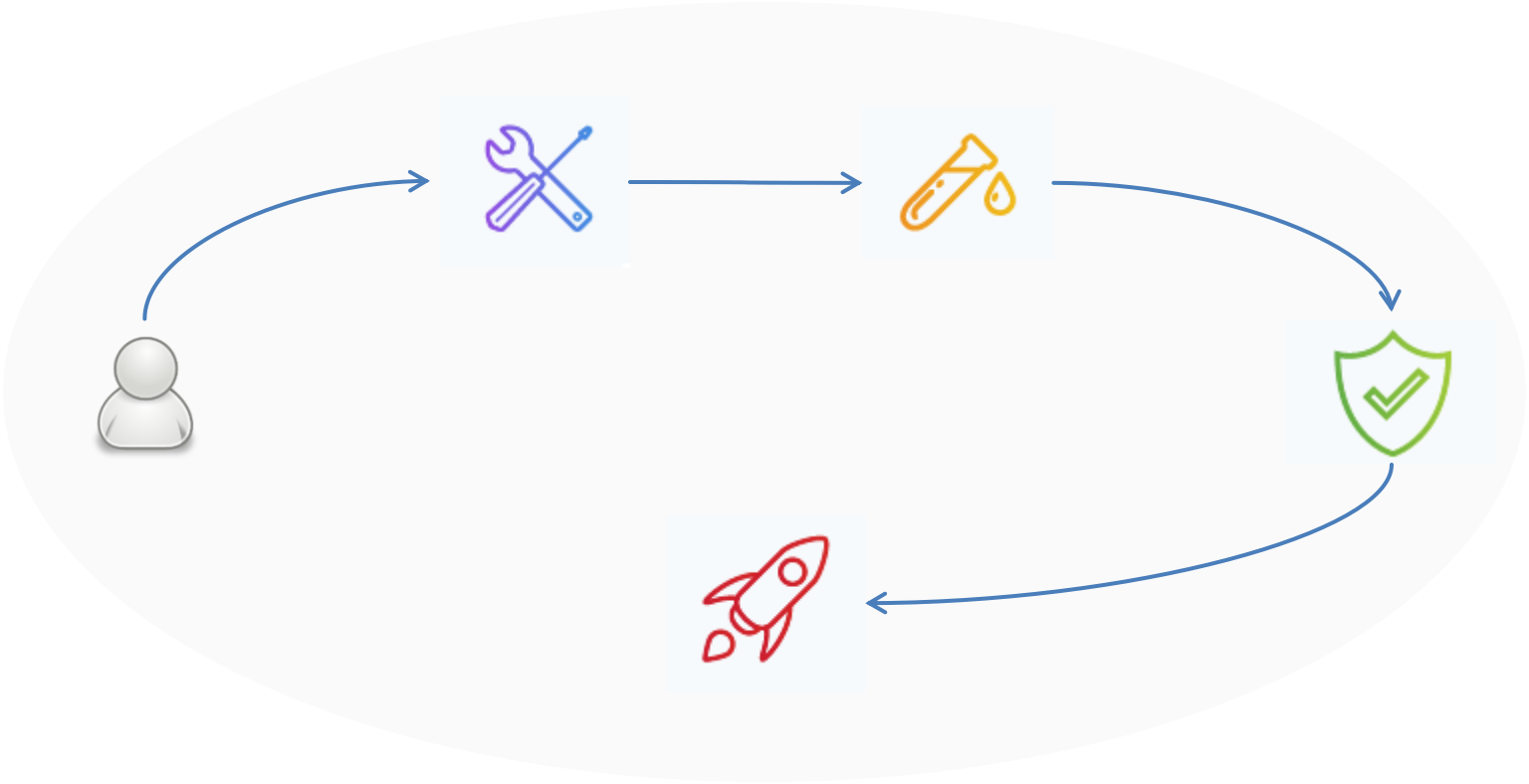
Program analyzers find software bugs

A program analyzer is software that analyzes other software to find bugs

Analyzers can prevent catastrophic defects in safety-critical software



I work on program analysis



How to build more reliable software while increasing developer productivity

I work on program analysis

- Integrating program-analysis techniques to combine their strengths

  [static analysis /\ (testing \/ verification), verification /\ testing, static analyses]

I work on program analysis

- Integrating program-analysis techniques to combine their strengths

  [static analysis /\ (testing \/ verification), verification /\ testing, static analyses]

- Making program analysis more widely applicable

  [ZK infrastructure, smart contracts, intelligent systems, heterogeneous software]

I work on program analysis

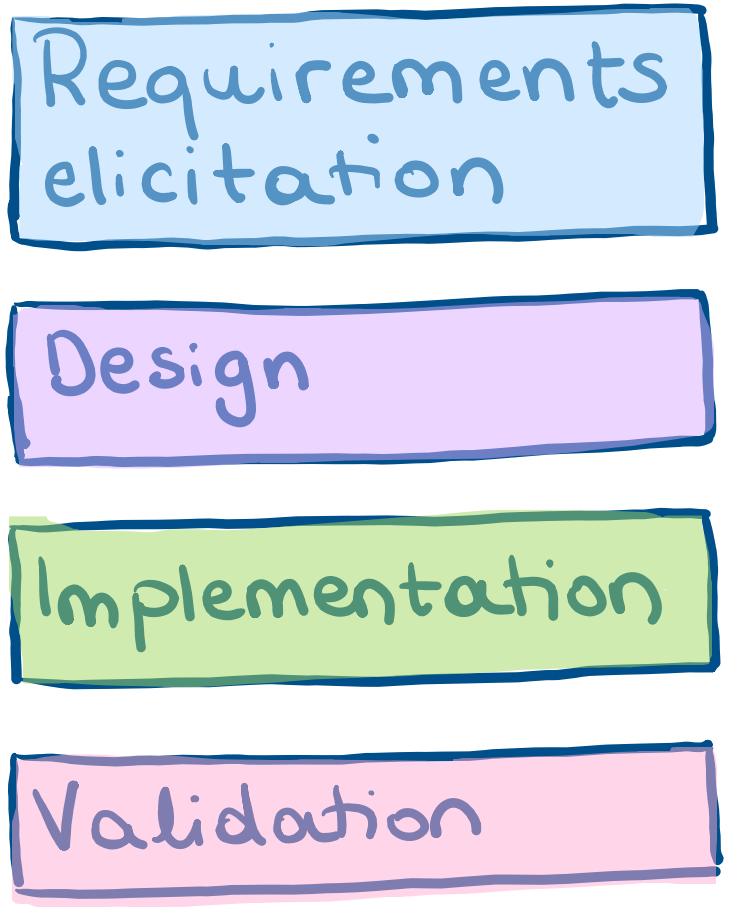
- Integrating program-analysis techniques to combine their strengths
  [static analysis /\ (testing \/ verification), verification /\ testing, static analyses]

- Making program analysis more widely applicable
  [ZK infrastructure, smart contracts, intelligent systems, heterogeneous software]

- Testing program analyzers for critical bugs
  [abstract interpreters, software model checkers, SMT solvers, Datalog engines]

About this course

The software development life cycle



Overview

- Automated validation techniques

- Theory and research applications
  - You will not be evaluated on the applications (unless stated otherwise during the lecture)
  - Applications will help in understanding the theory

Topics

- Abstract Interpretation
- Unsound Static Analysis
- Bounded Model Checking
- Symbolic Execution
- Dynamic Symbolic Execution
- Fuzzing
- Specification-Based, Differential, and Metamorphic Testing
- Specification Inference

Administration

Important dates:

- See TUWEL

Exercise sessions:

- Registration on TISS
- Exercise sessions start sharp (s.t.)

Tests:

- Registration on TISS

Lecture recordings / livestream:

- See TUWEL

Grading:

- See TUWEL

Test results:

- See TUWEL

Questions

1. Ask now :)
2. Check if the answer is already on TISS or TUWEL
3. If the question is relevant for many, ask on the TUWEL forum
3. If none of the above, email aseinfo@se.tuwien.ac.at

Terminology

Program analysis

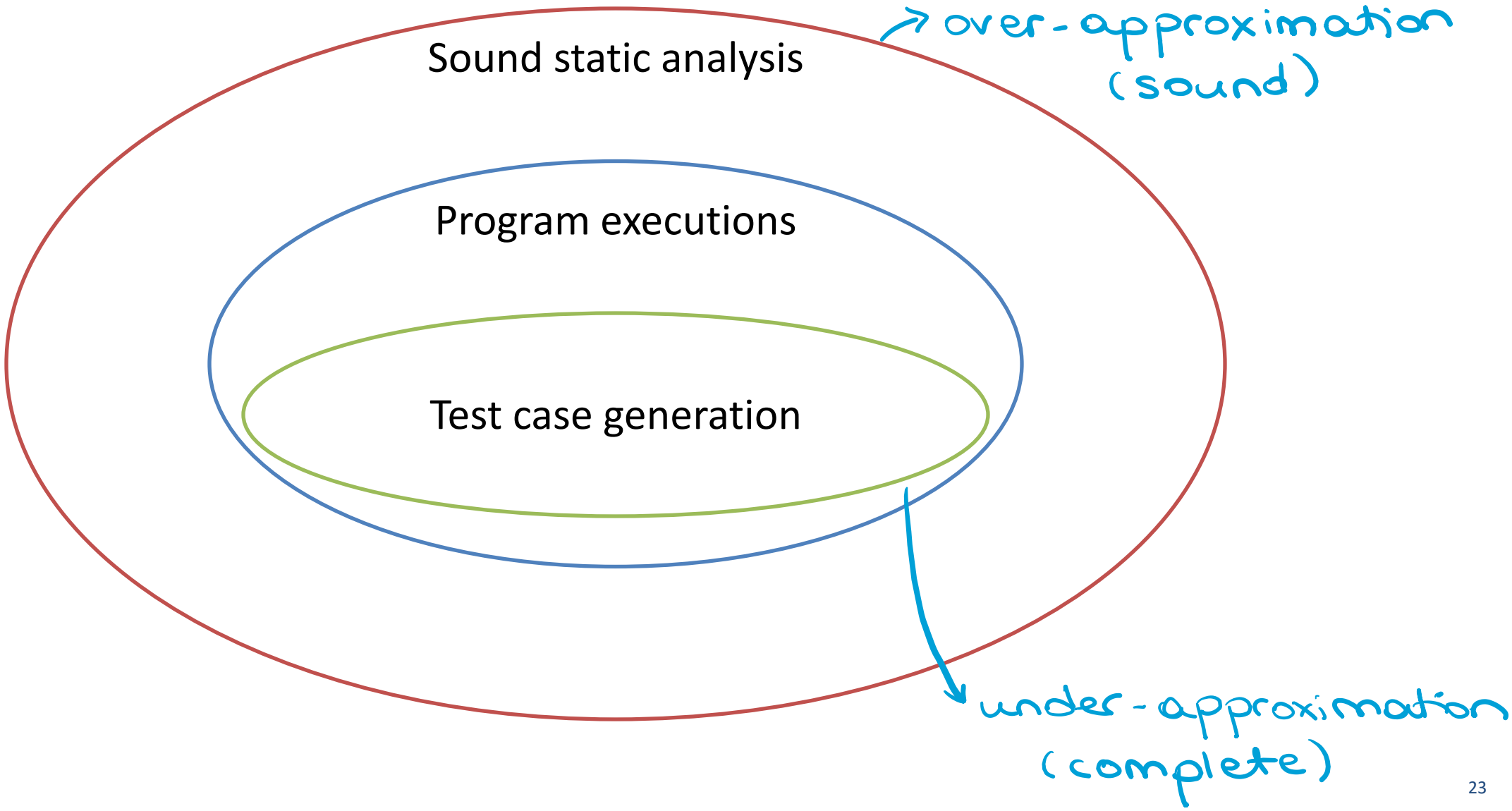
- Static analysis automatically inspects code (without running it) and emits warnings about potential bugs

- Test case generation repeatedly executes a program with different, automatically generated inputs with the goal of finding bugs

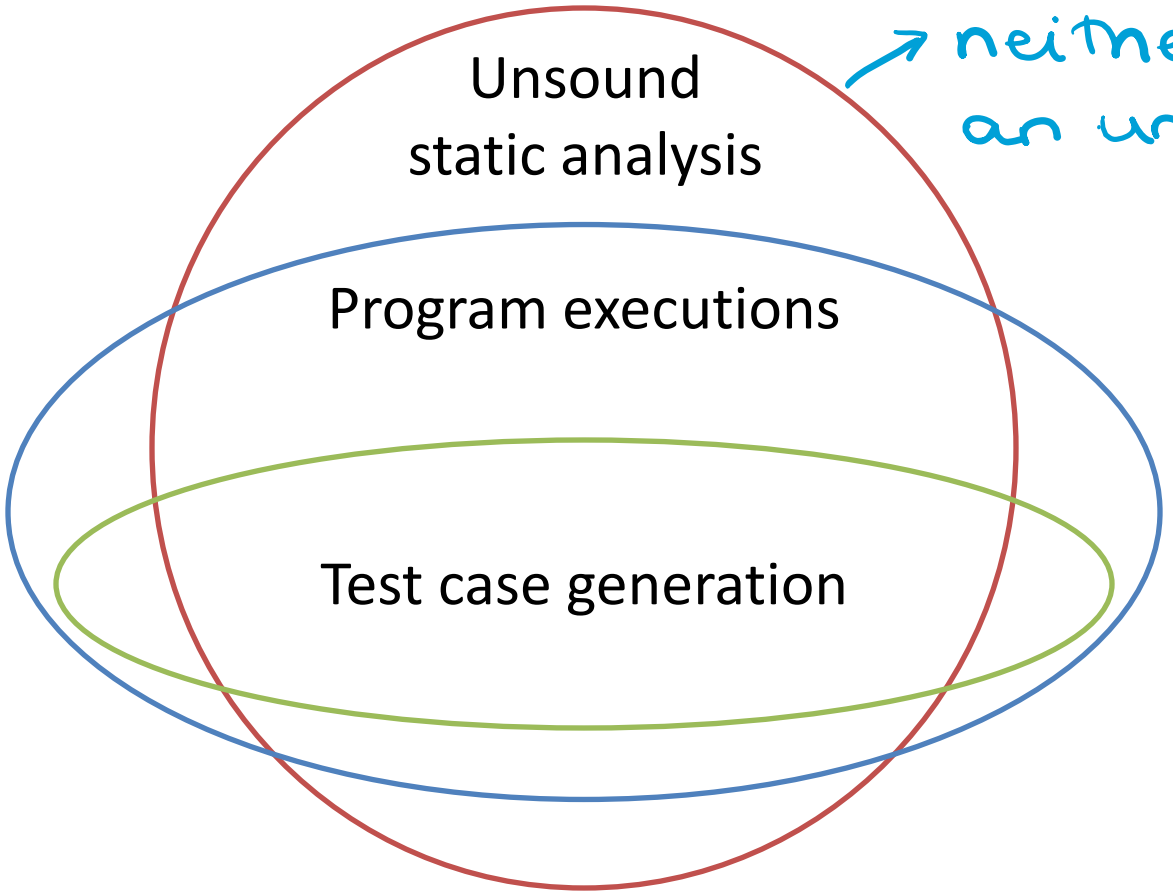
Terminology in program analysis

- Soundness means no false negatives
  - A false negative is a missed bug in the code by the program analysis, i.e., the analysis did not emit a warning for an existing bug
- Completeness means no false positives
  - A false positive is a warning emitted by the program analysis for code that is correct
- Precision means trying to minimize incompleteness

Static analysis and test case generation

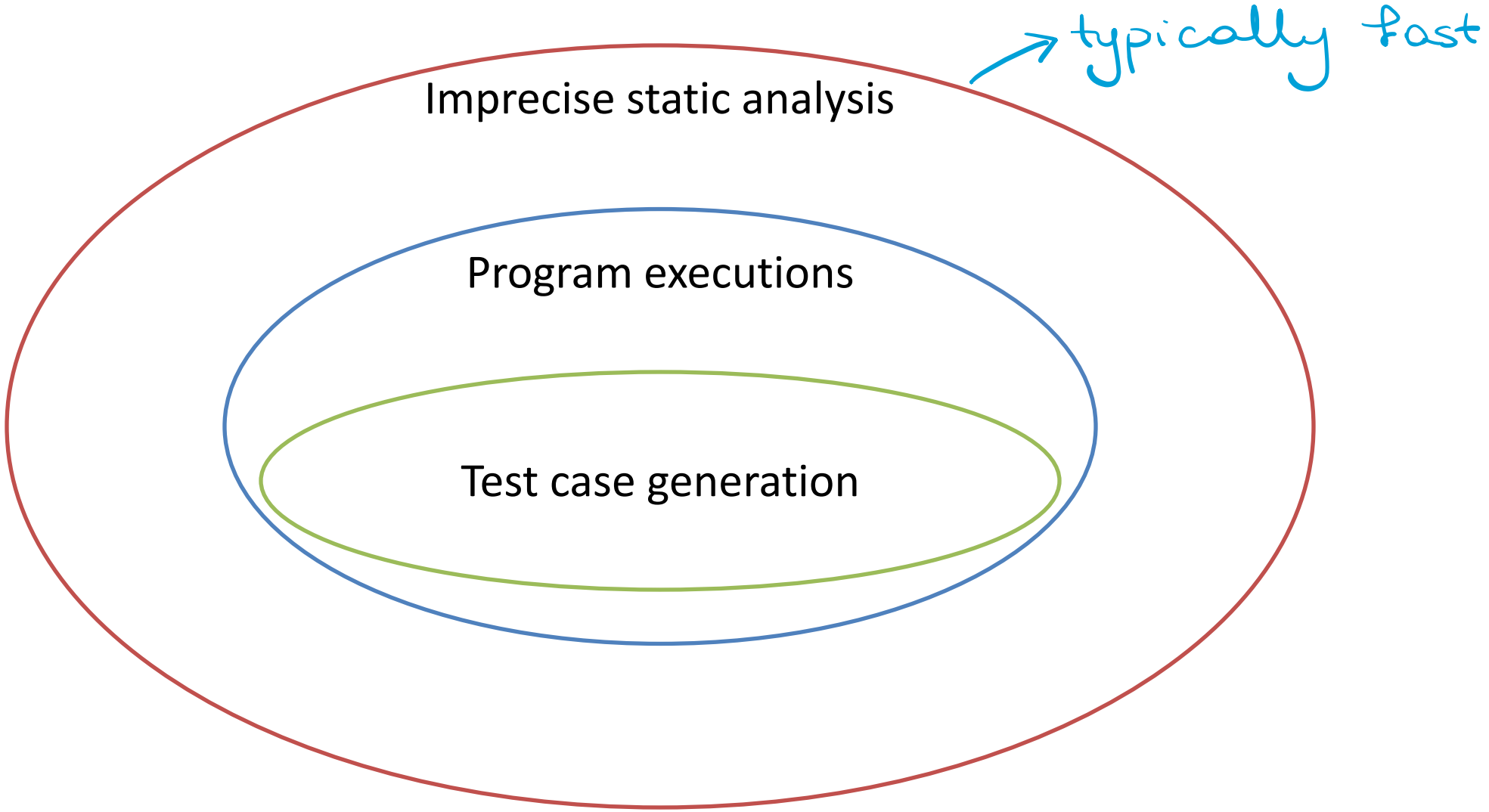


Unsoundness in static analysis

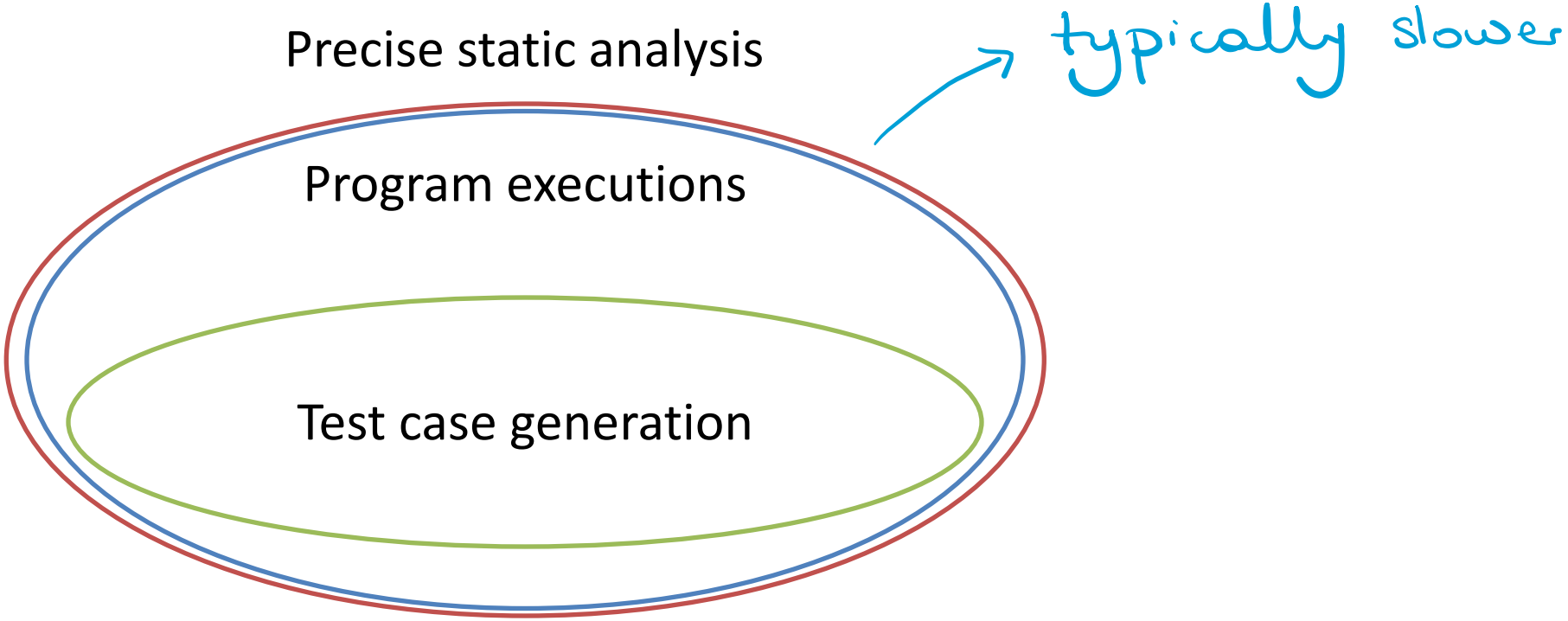


neither an over- nor an under-approximation

Imprecision in static analysis



Precision in static analysis



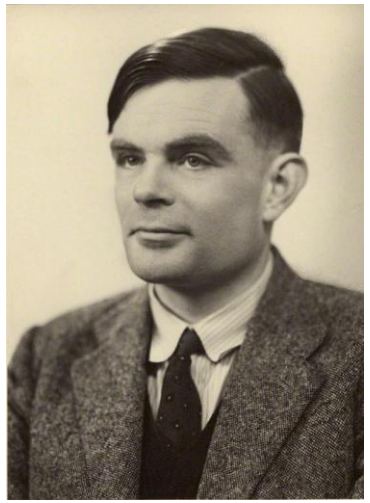
Abstract interpretation: Theory

Building a static analysis: Question

- Can you build an automatic static analyzer that takes as input an arbitrary program and an arbitrary property such that if the analyzer answers:
  - "Yes", then it is certain that the property holds
  - "No", then it is certain that the property does not hold

Building a static analysis: Question

- Can you build an automatic static analyzer that takes as input an arbitrary program and an arbitrary property such that if the analyzer answers:
  - "Yes", then it is certain that the property holds
  - "No", then it is certain that the property does not hold



Alan Turing

No, the problem is undecidable.

Building a static analysis: New question

- Can you build an automatic static analyzer that takes as input an arbitrary program and an arbitrary property such that if the analyzer answers:
  - "Yes", then it is certain that the property holds
  - "No", then it is ~~certain that the property does not hold~~ unknown if the property holds or not

Trivial static analysis

```
StaticAnalyzer(Program, Property)
{
  return “No”;
}
```

Building a static analysis: Challenge

The challenge is to build a static analyzer that can answer “Yes”

for as many programs that satisfy the property

Static analysis: Cool facts

- Can automatically prove interesting properties such as
  - Absence of null pointer dereferences
  - Assertions at a program point
  - Termination
  - Absence of data races
  - Information flow, …
- Can “run” the program without concrete inputs
  - It can abstractly simulate the program execution from any point
- No need for manual annotations

Abstract interpretation: Terminology

- Abstract domains are standard abstractions for building abstract interpreters
  - They abstract program data, basic operations, and control
- An abstract program state is a map from program variables to elements in the abstract domain
- An abstract transformer describes the effect of statement and expression evaluation on an abstract state

Abstract interpretation: Terminology

Abstract interpretation involves three steps:

1. Selecting an abstract domain

2. Defining abstract transformers

3. Iterating the transformers over the domain to obtain an over-approximation
   - We iterate until we reach a fixed point, which is the over-approximation of the program

4. Check the properties

If the over-approximation is correct, then the program must be correct

Example

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

- It will take far too long to enumerate all program executions and check if the assertion holds for all of them
- Instead, let us do some over-approximation so that we can reduce the space of what we need to enumerate

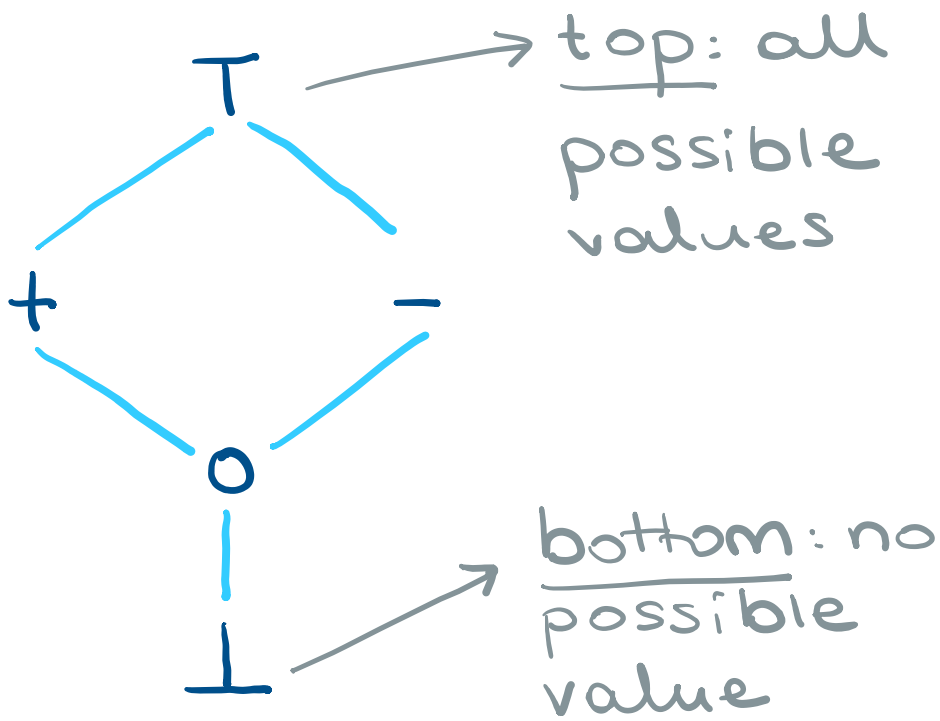
Example – Step 1: Select abstraction

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

- Let's pick the sign abstract domain



- + represents all positive numbers and 0
- – represents all negative numbers and 0

Example – Step 1: Select abstraction

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

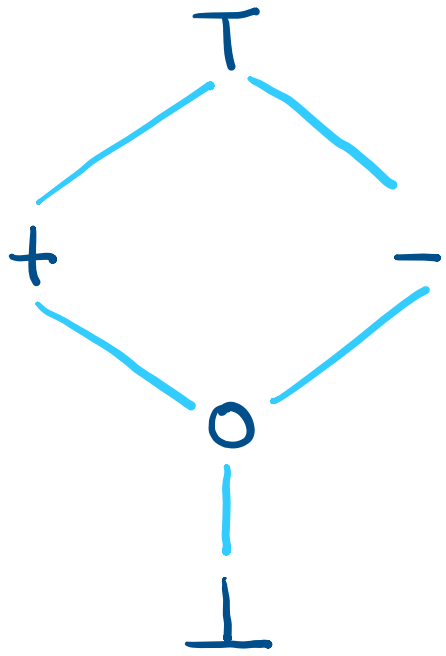
pc: 2

x : +

y : ⊥

i : ⊤

example abstract state in the sign domain

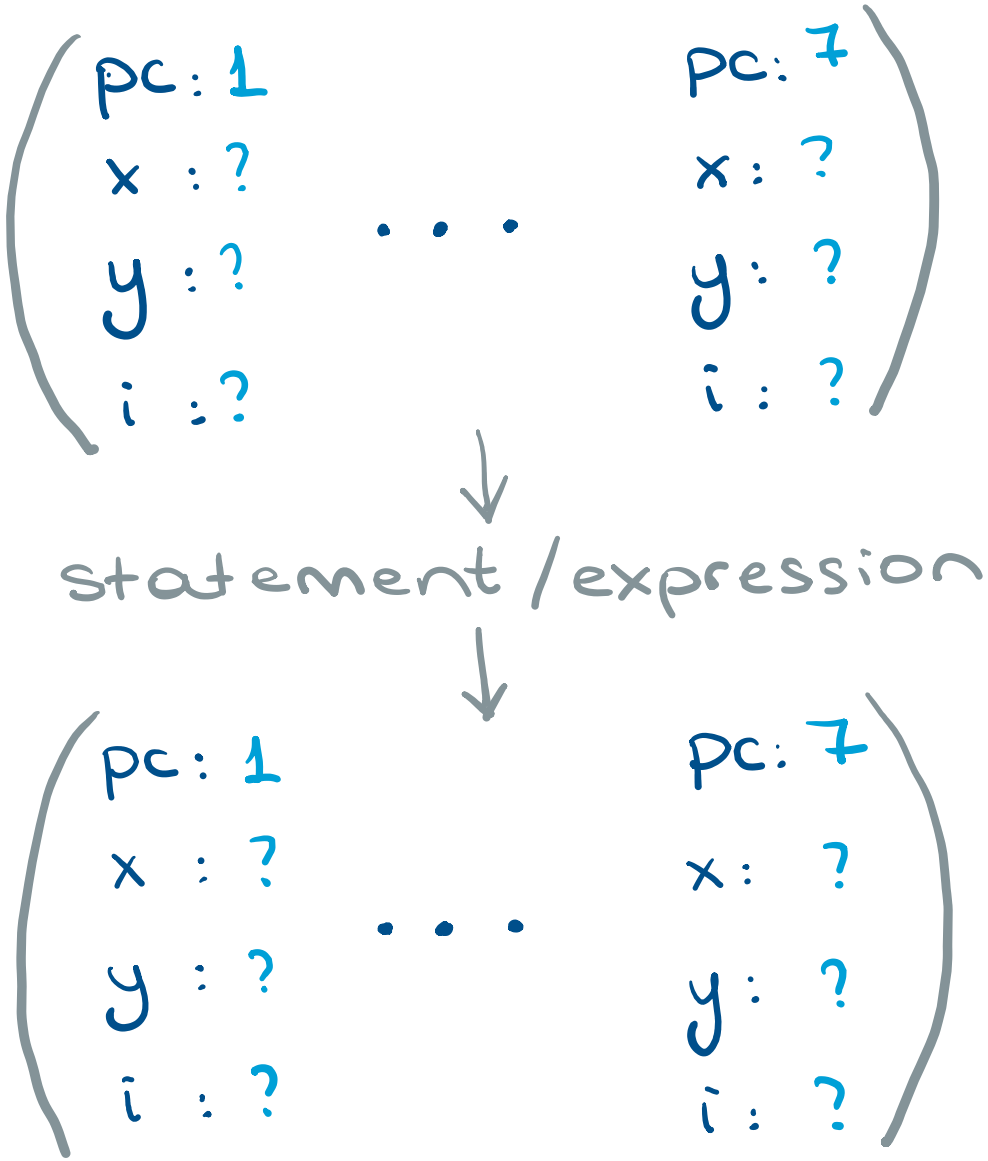


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Abstract transformers: Important point

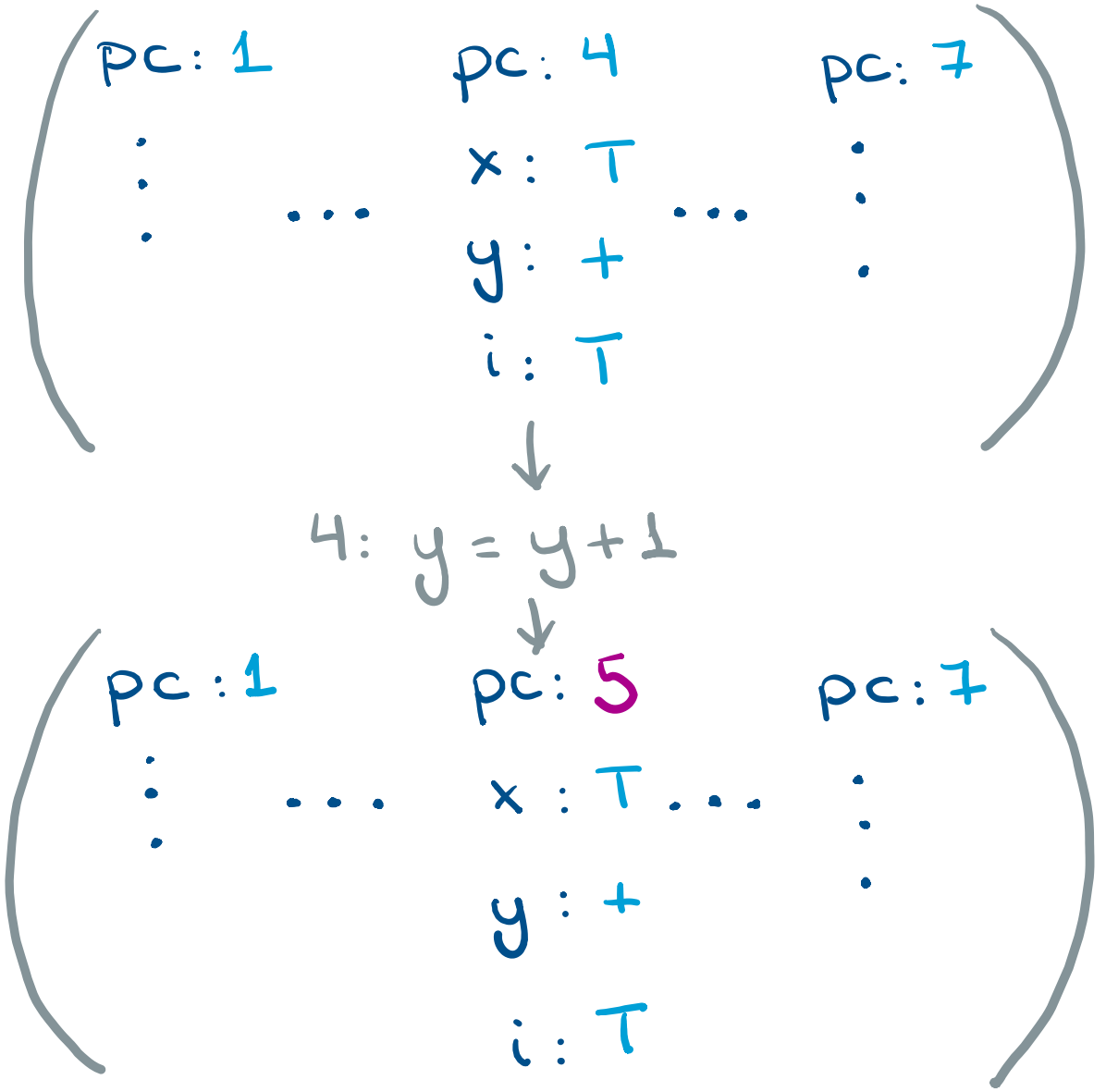
- Abstract transformers are defined per programming language, not per program
- They essentially define the new abstract semantics of the language
- Any program in the programming language can use the same transformers

Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i – 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

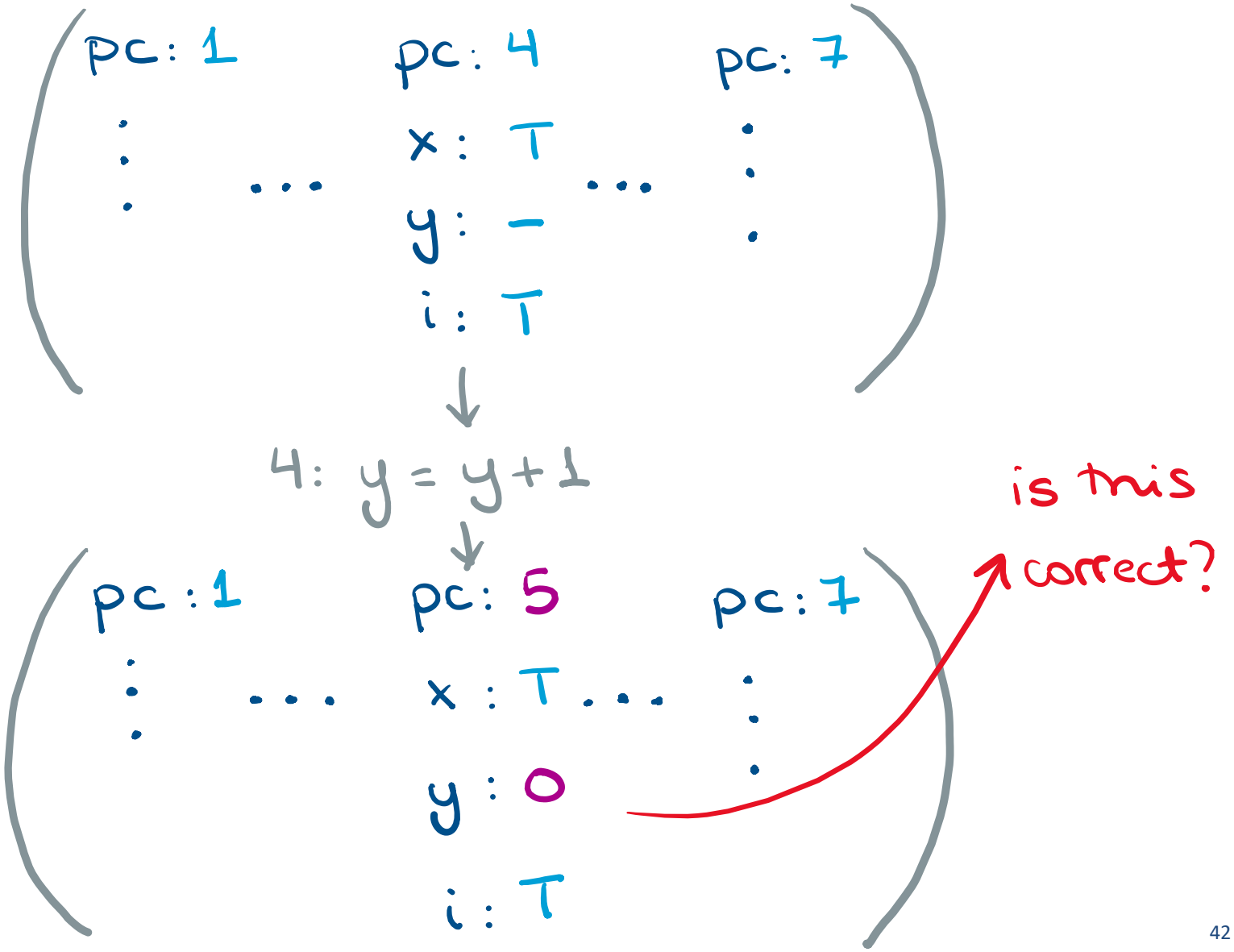


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

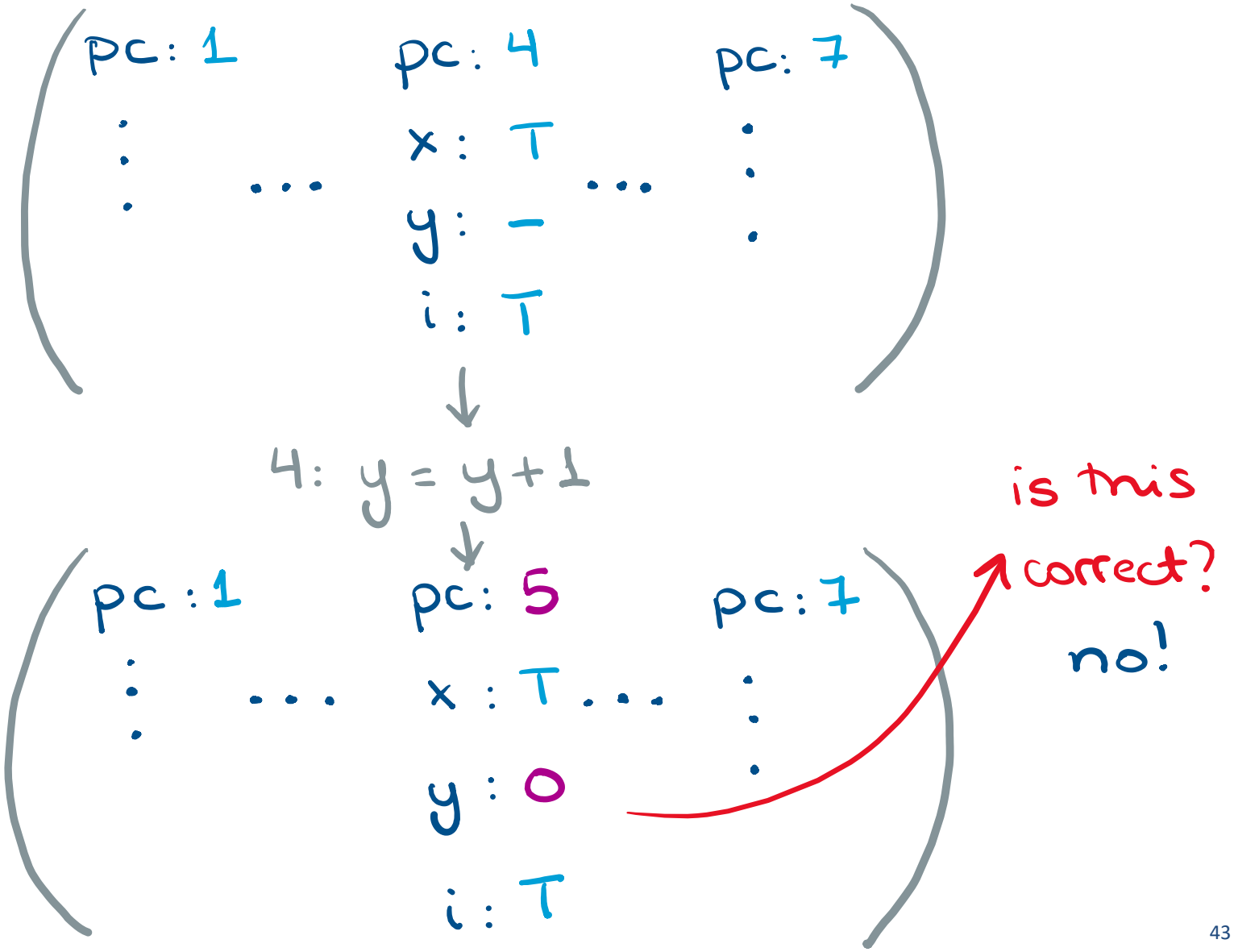


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Abstract transformers: Correctness

- A correct abstract transformer should always produce results that are a superset of what a concrete transformer would produce

- In other words, a correct abstract transformer is a sound transformer

Abstract transformer: Unsoundness

This abstract state

pc: 4
x: T
y: –
i: T

represents infinitely many concrete states, including

pc: 4
x : 1
y : -3
i : 2

If we perform $y = y + 1$ on this concrete state,

we get

pc: 5
x : 1
y : -2
i : 2

Abstract transformer: Unsoundness

However, the abstract transformer produced an abstract state

pc: 5
x: T
y: 0
i: T

This abstract state does not represent any state where $y = -2$

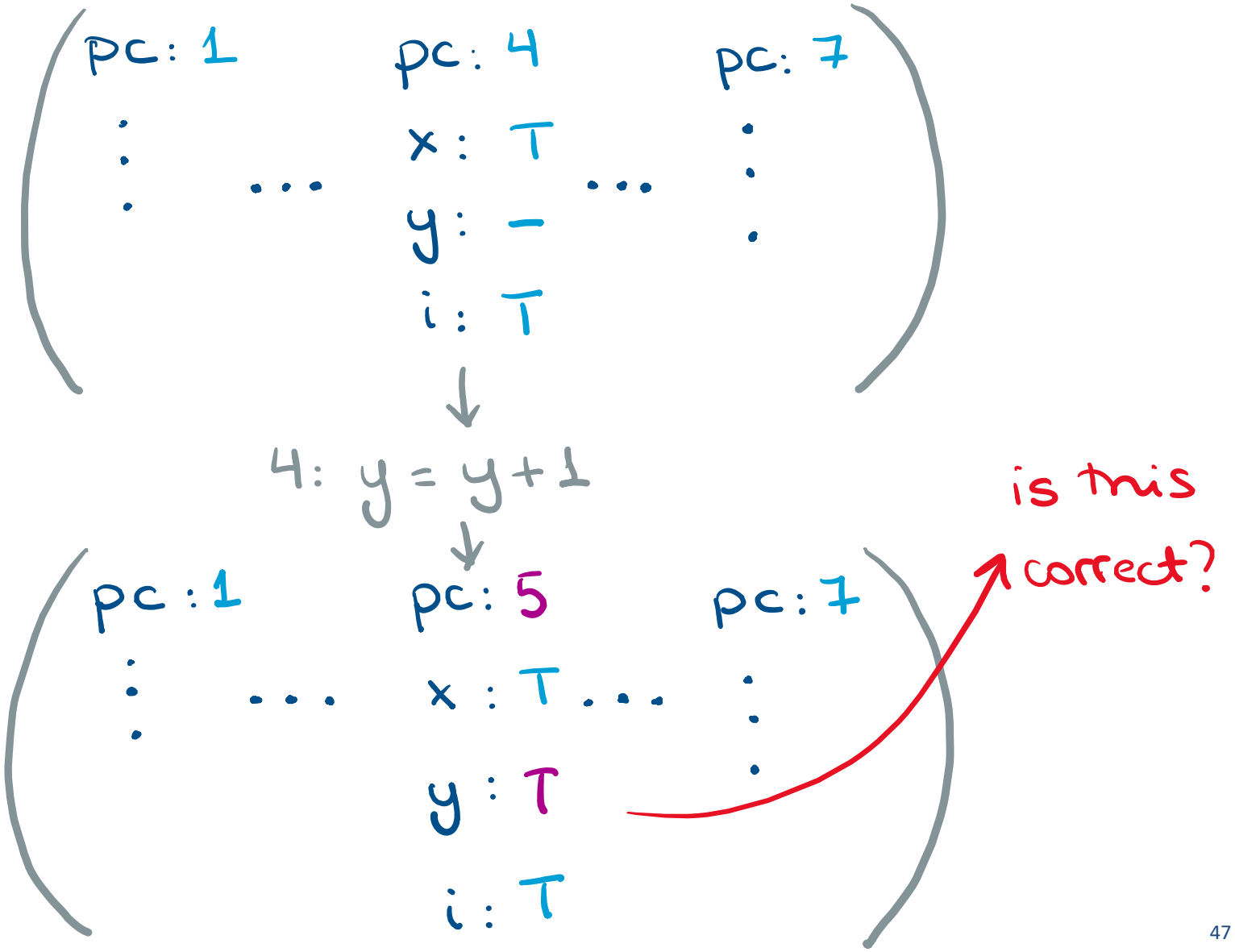
This means that the abstract transformer is unsound

Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

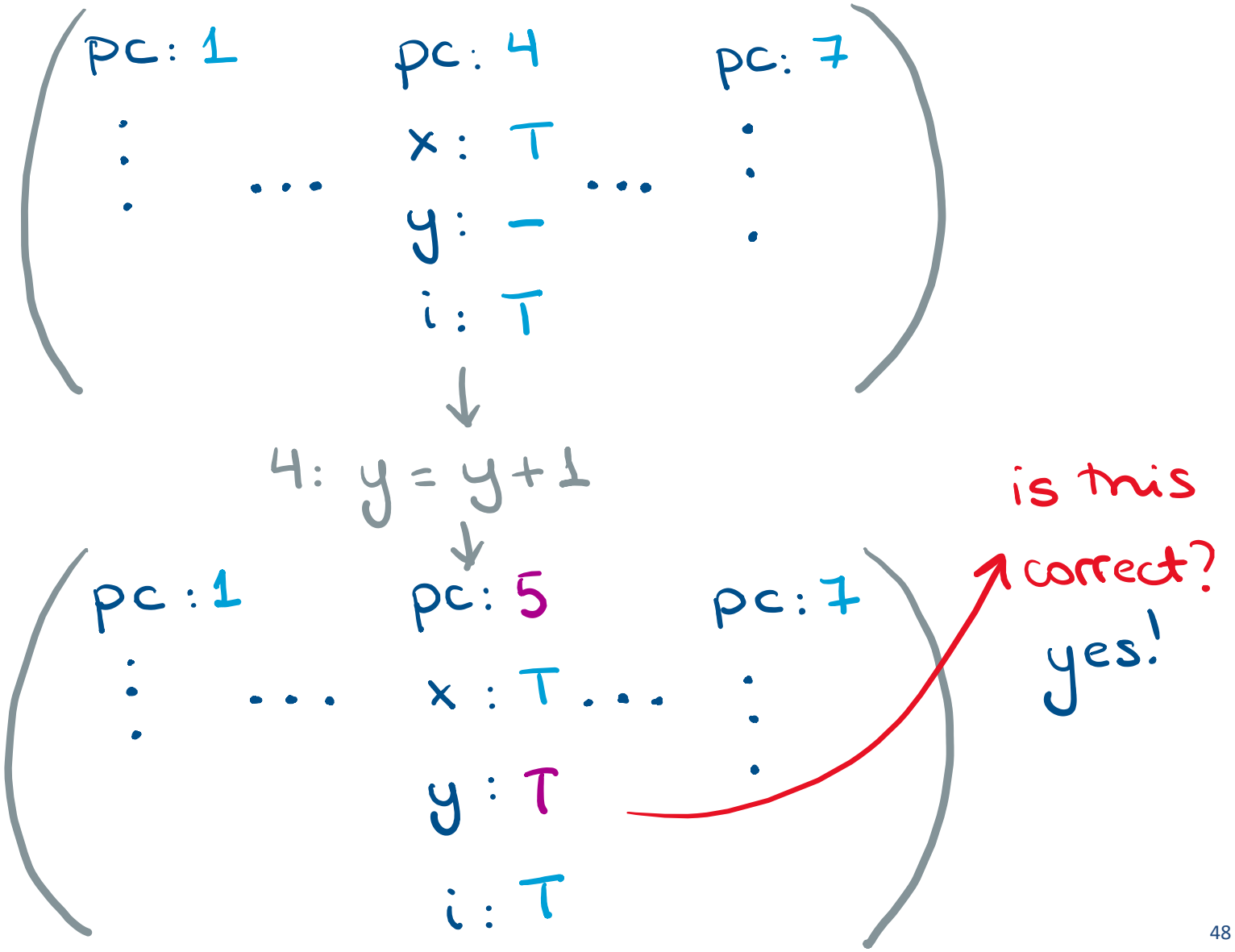


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

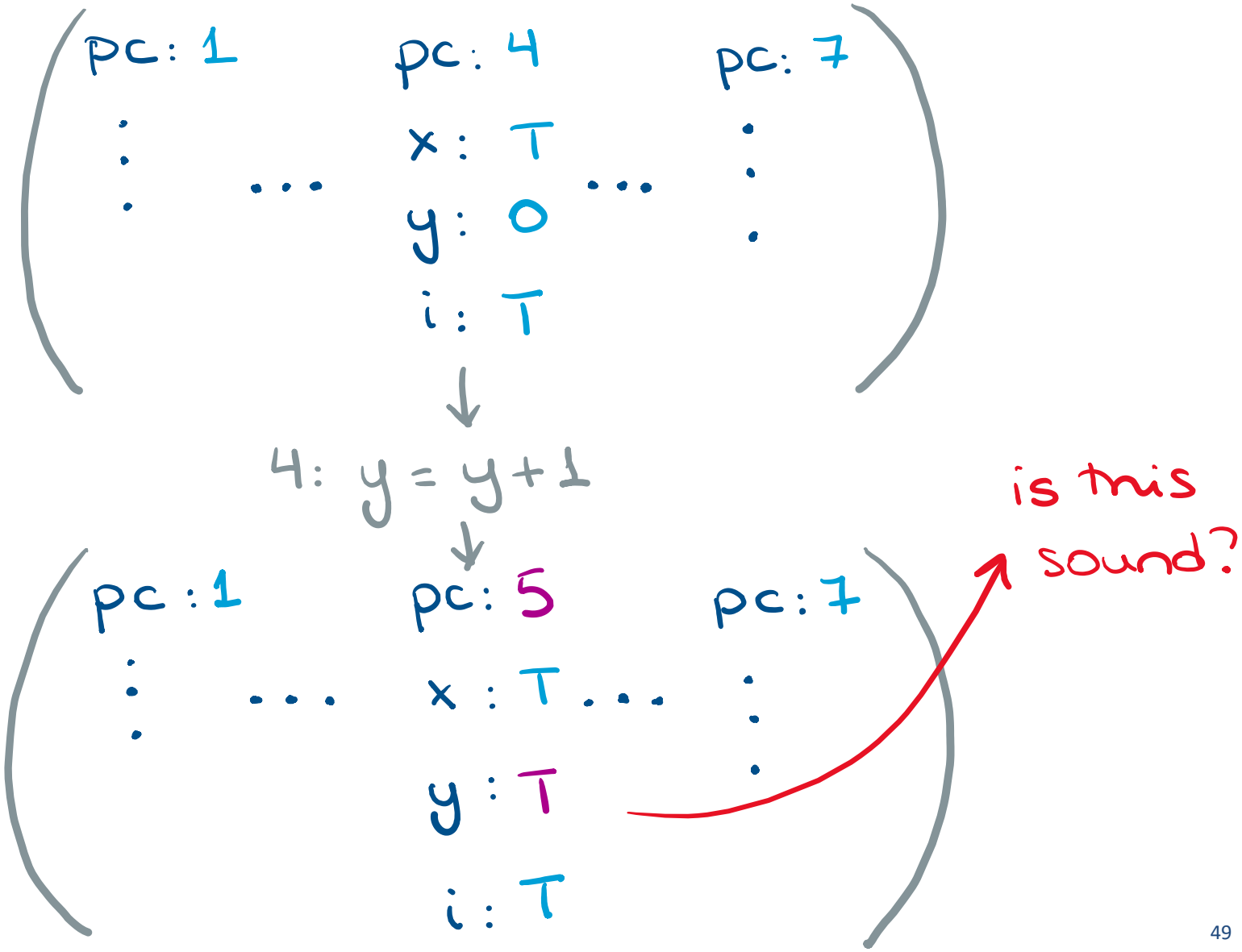


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

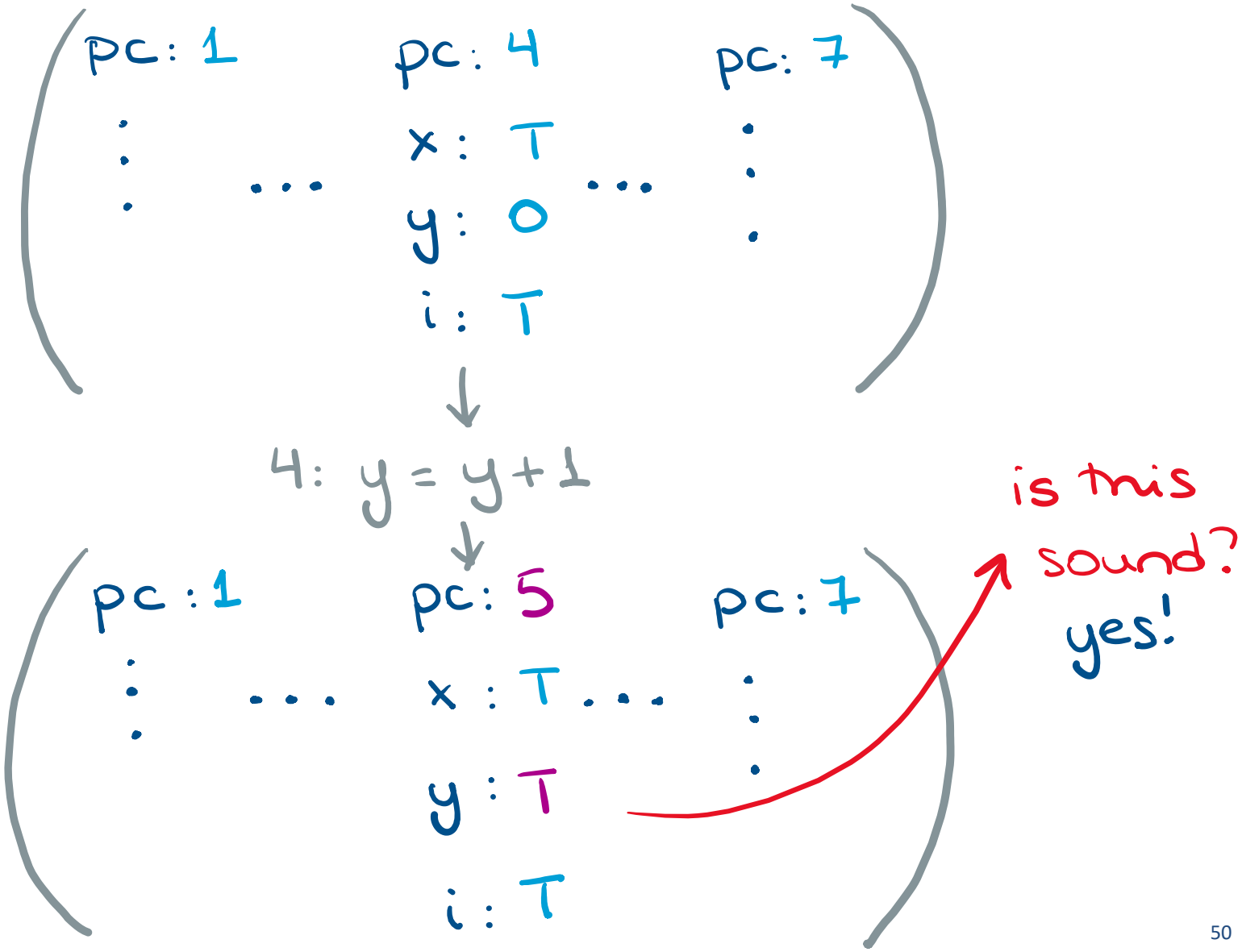


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

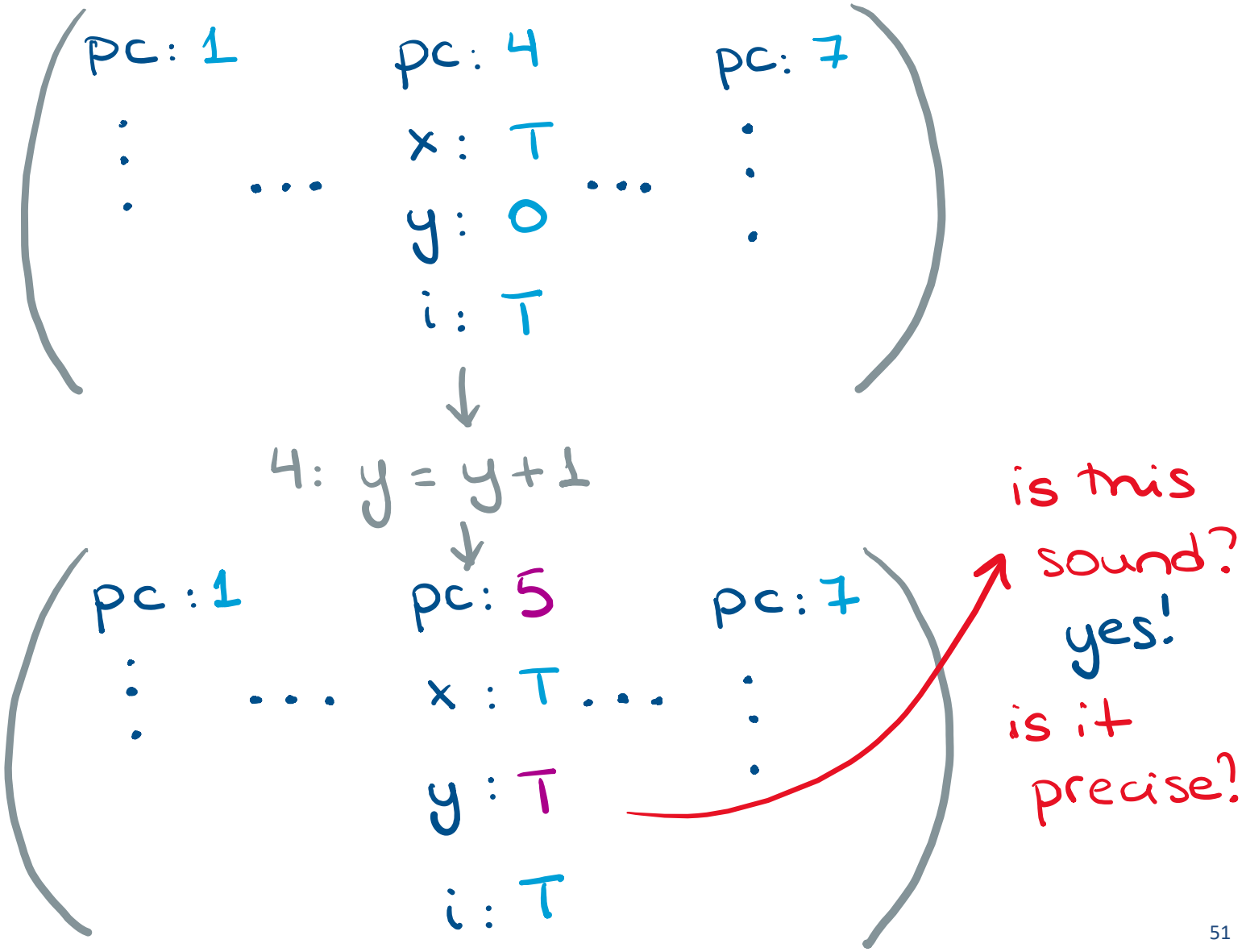


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

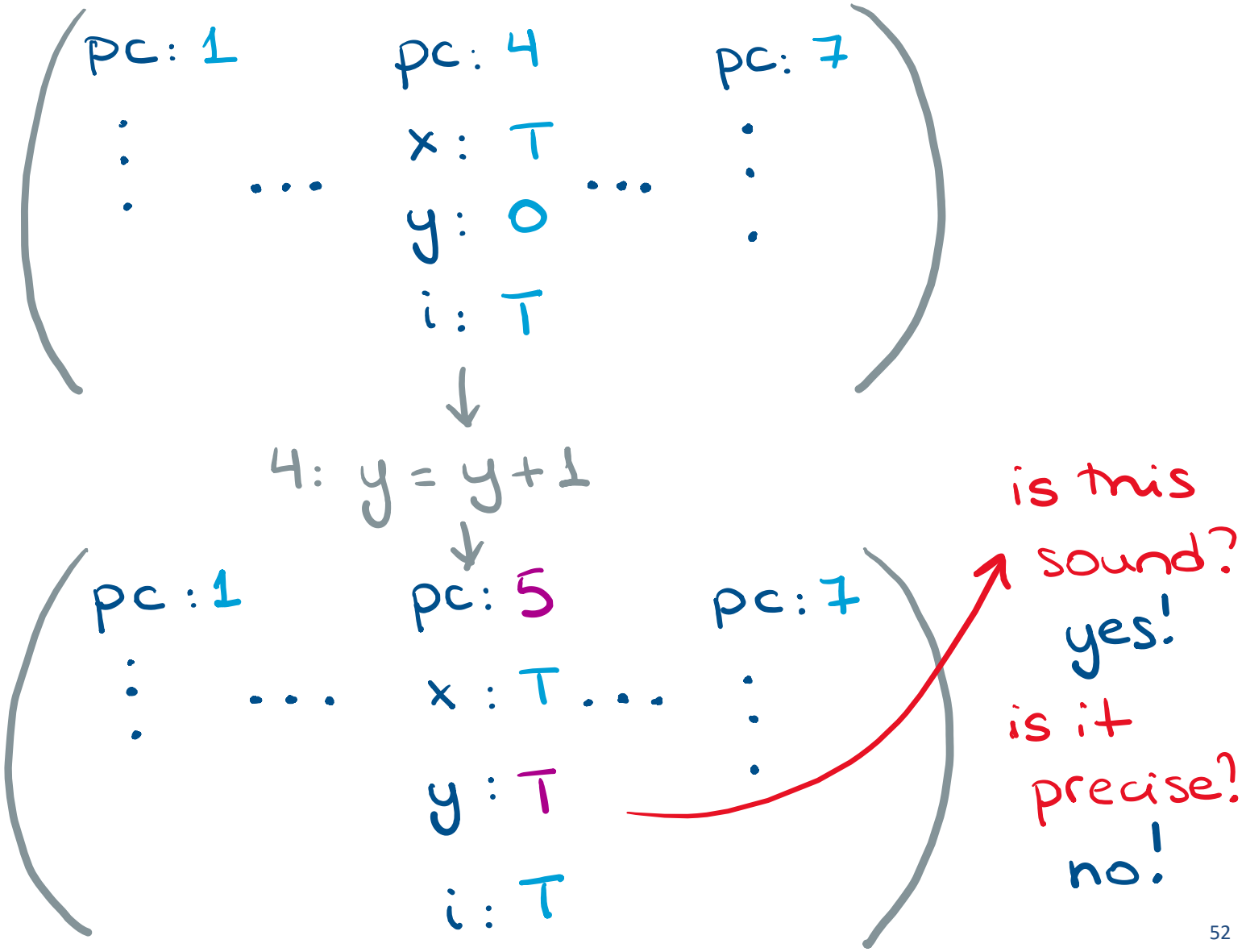


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Abstract transformer: Imprecision

This abstract state

pc: 4
x : T
y : 0
i : T

represents infinitely many concrete states, including

pc: 4
x : 1
y : 0
i : 2

If we perform $y = y + 1$ on any of these concrete states, we will always get states where y is positive, such as

pc: 5
x : 1
y : 1
i : 2

Abstract transformer: Imprecision

However, the abstract transformer produces an abstract state where y can have any value, such as

pc: 5
x : 1
y : -3
i : 2

There exist sound transformess that produce fewer results

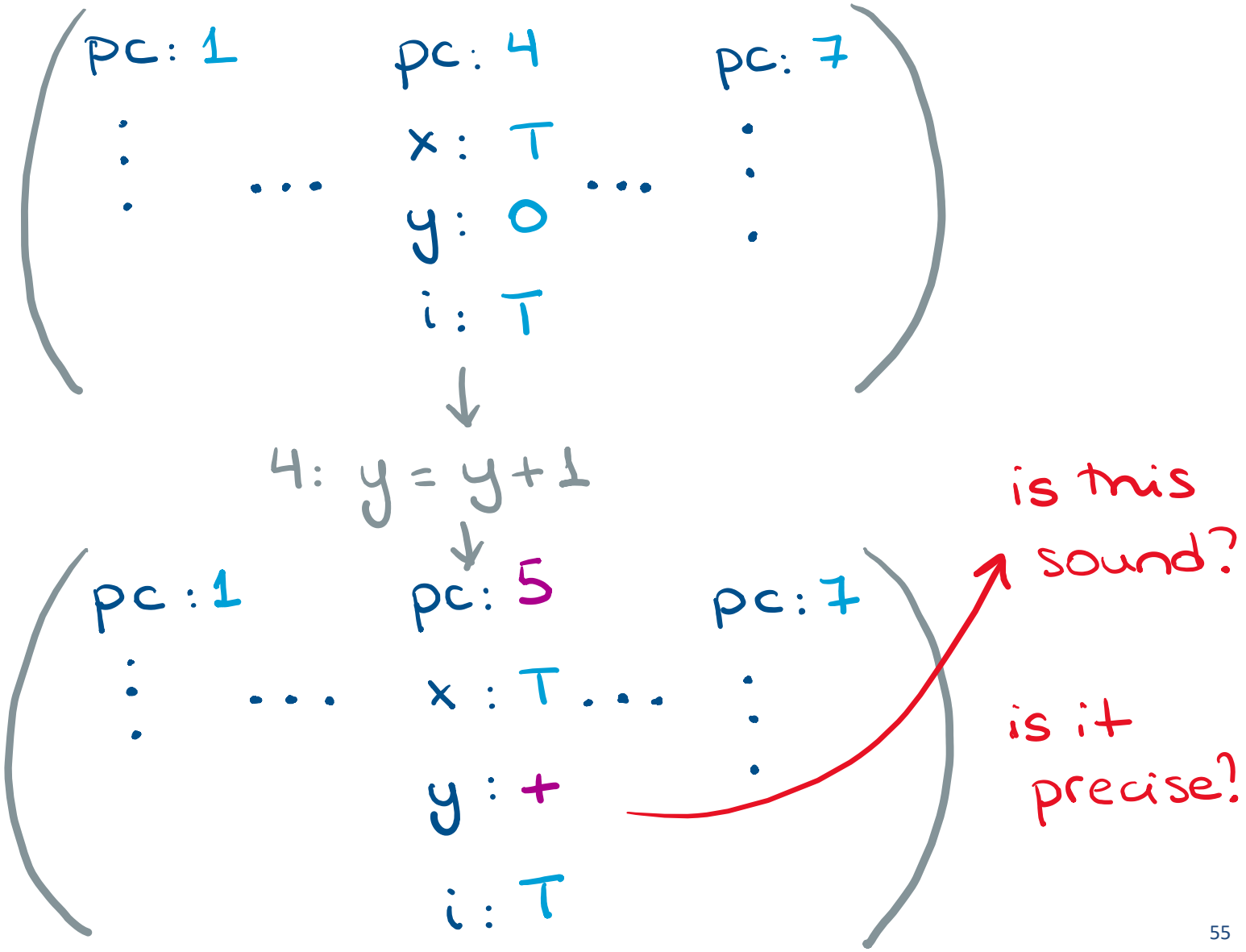
This means that the abstract transformer is imprecise

Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```

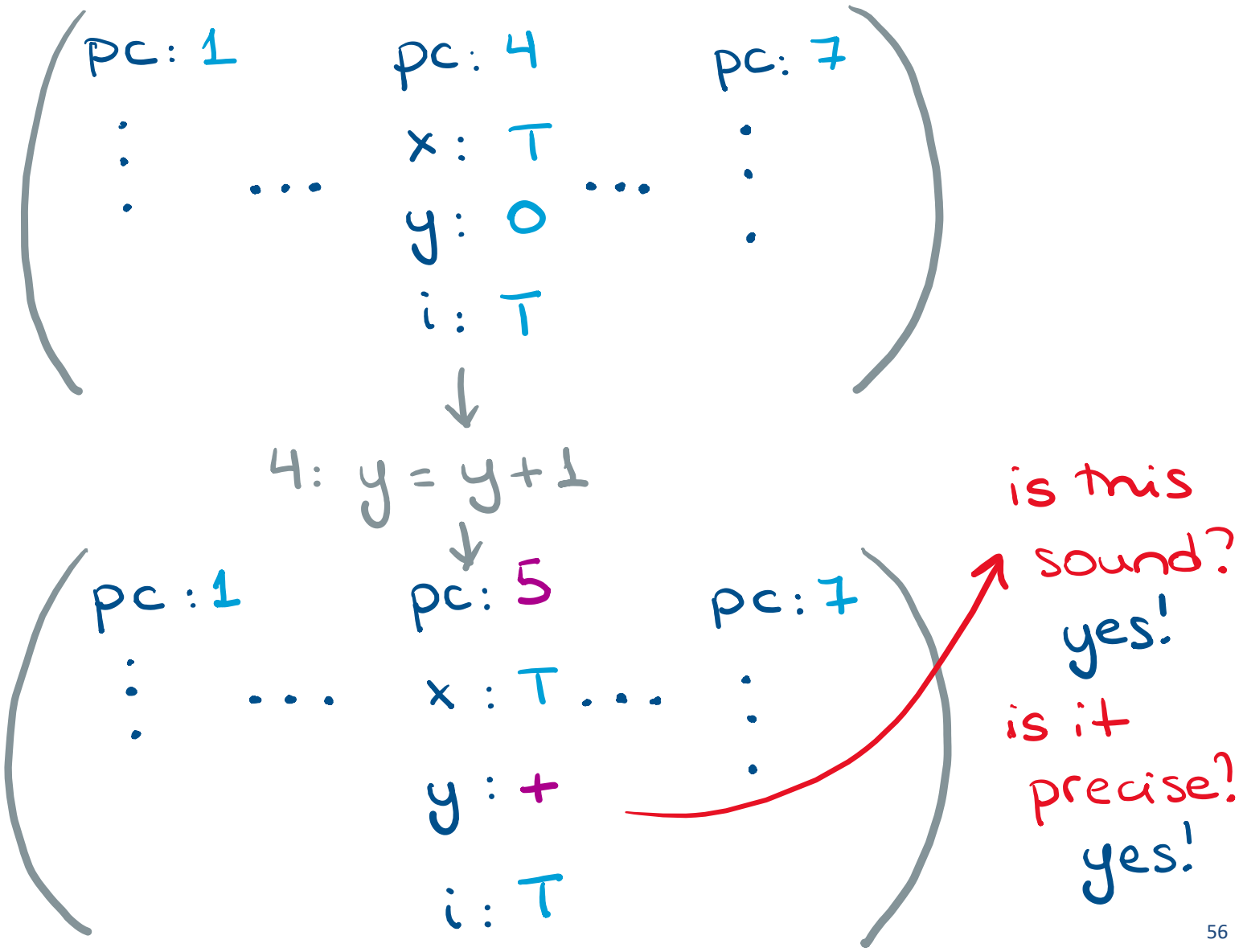


Example – Step 2: Define transformers

```
void Foo(int i) {
  1: int x = 5;
  2: int y = 7;

  3: if (i >= 0) {
  4:   y = y + 1;
  5:   i = i - 1;
  6:   goto 3;
     }

  7: assert 0 <= x + y;
}
```



Abstract transformers: Properties

- It is easy to be sound and imprecise
  - Simply output T
- It is desirable to be both sound and precise
- If we lose precision, it needs to be clear why and where
  - Sometimes, computing the most precise (and sound) transformer, called the best transformer, is impossible
  - For efficiency reasons, we may sacrifice some precision